

Bugs: Their Kinds, and Their Lives

Five families of defects and the thirteen states every bug can pass through.

UNIVERSITY QUESTIONS · SIX APPEARANCES COMBINED

1.7 “Classify different types of bugs based on SDLC” — May 17, May 19, Dec 19

1.8 “Explain the Life Cycle of a Bug and its defect states” — Dec 17, Dec 18, Dec 19

1.7 · THE CASE FOR A TAXONOMY

A bug report without a category is a task without an owner.

Bug tracking systems classify defects by severity and priority by default. But your textbook recommends adding a third classifier the NATURE of the bug because it streamlines the assignment of bug-fixing tasks to the responsible teams: a performance defect goes to one specialist, a security defect to a very different one.

Five families: Functional · Performance · Usability · Compatibility ·
Security

Functional & Performance

01 · FUNCTIONAL DEFECTS

Errors identified when the software's behaviour is not compliant with the functional requirements the product does the wrong thing, or fails to do the required thing.

FIELD EXAMPLE

An e-commerce site's search returned nothing when a user typed a product ID though the requirements stated search must work by both product name AND ID.

Found via: Functional testing

02 · PERFORMANCE DEFECTS

Defects bound to the software's speed, stability, response time, and resource consumption the product does the right thing, but too slowly, too unreliably, or too expensively.

FIELD EXAMPLE

A system's response time measured X times longer than the limit stated in the requirements.

Found via: Performance testing

Usability & Compatibility

03 · USABILITY DEFECTS

Defects that make an application inconvenient to use, hampering the user's experience the product works, but fights its own users.

FIELD EXAMPLE

A content layout that's difficult to scan or navigate; an overly complex signup procedure. Testers and UX designers validate against usability requirements and the Web Content Accessibility Guidelines (WCAG).

Found via: Usability testing

04 · COMPATIBILITY DEFECTS

The application doesn't perform consistently on particular hardware, operating systems, browsers, devices or when integrated with certain software or network configurations.

FIELD EXAMPLE

A car-insurance claim estimation app failed to meet requirements on Android 8.0 and 8.1 defects in font size, content alignment, and the scroll bar.

Found via: Compatibility testing

Security Defects

“ *Security defects are the weaknesses allowing for a potential security attack.*

— as framed by a penetration testing consultant & certified ethical hacker, quoted in your text

THE MOST FREQUENT SECURITY DEFECTS FOUND IN PRACTICE

- **Encryption errors**
Data protected badly — or not at all.
- **XSS vulnerabilities**
Attacker's script runs inside other users' browsers.
- **Weak authentication**
The door exists, but the lock is trivial.
- **SQL injection susceptibility**
Attacker's input executed as database commands.
- **Buffer overflows**
Input overruns memory it was never meant to touch.
- **Role-based access logic errors**
The right lock on the wrong doors.

Found via: security testing & penetration testing and we're about to meet the most famous security defect of the decade.

The Five Families, Side by Side

Family	The product...	Discovered via
Functional	...does the wrong thing, or misses a required behaviour.	Functional testing
Performance	...does the right thing — too slowly, unstably, or expensively.	Performance testing
Usability	...works, but is inconvenient and fights its users.	Usability testing (vs. WCAG)
Compatibility	...behaves inconsistently across devices, OS, browsers, networks.	Compatibility testing
Security	...contains weaknesses that allow a potential attack.	Security / penetration testing

CASE STUDY

Heartbleed, 2014

A security defect — one missing bounds check — that silently exposed the internet's secrets for over two years.

OpenSSL is the open-source encryption library that secured a huge share of the world's HTTPS websites the padlock in your browser. In early 2012, a routine new feature (the TLS 'heartbeat' extension) shipped with one flaw: the code trusted the sender's claim about how much data to echo back, without checking it. An attacker could claim '64,000 bytes, please' after sending almost nothing and the server would obligingly reply with 64 KB of whatever happened to be in its memory: session cookies, passwords, even its own private encryption keys.

CLASSIFY IT

A textbook security defect specifically a buffer over-read, first cousin of the buffer overflow on our list two slides ago.

2+ yrs

the defect sat undetected in production, worldwide

SECURE SERVERS EXPOSED
~17%

SIZE OF THE FIX
A few lines

Disclosed April 7, 2014 as CVE-2014-0160. Within days, Canada's tax agency confirmed roughly 900 citizens' social insurance numbers stolen via the flaw during tax season.

ROOT CAUSE

Trust without verification — in code and in process

The code trusted an attacker-controlled length field a missing bounds check. But the deeper cause was procedural: OpenSSL, securing much of the internet, was maintained by a tiny team around one full-time developer on roughly \$2,000 a year in donations. There was no budget for the security testing and code review the whole world was silently depending on.

The aftermath created the Core Infrastructure Initiative major tech companies finally funding the security testing of critical open-source code.

1.8 · NOW: ONE BUG'S WHOLE LIFE

The Defect Life Cycle is the specific set of states a defect or bug goes through in its entire life.

Its purpose: to coordinate and communicate the current status of a defect as it changes hands between assignees — making the defect-fixing process systematic and efficient. The DEFECT STATUS is the present state the bug is currently in; its goal is to precisely convey progress, so everyone can track where each defect actually stands. The exact number of states varies from project to project Fig. 1.8.1 covers all the possible ones.

The Main Path: New → Closed



THE LOOP · REOPENED

If the bug persists after the fix, the tester sets it to Reopened — and the bug travels the life cycle all over again.

The Four Side Exits

Not every logged defect deserves the full journey — four states let a bug leave the main path.

DUPLICATE

The defect is repeated — it corresponds to the same underlying bug already logged by someone else.

REJECTED

The developer judges that the reported defect is not a genuine defect at all.

DEFERRED

Real — but not of prime priority; expected to be fixed in the next release instead of this one.

NOT A BUG

The reported behaviour does not affect the functionality of the application.

Exam tip: main path (8 states) + loop (Reopened) + side exits (4 states) = the 13 states of Fig. 1.8.1.

Three Questions Before Anyone Writes Code



IF THE DEFECT SURVIVES ALL THREE QUESTIONS

It's assigned to a developer (status: In-Progress) → the code is fixed (Fixed) → the tester retests. Test passes? CLOSED. Test fails? RE-OPEN — back to the developer, and around the loop again.

Key Takeaways

THE BUG THAT CAME BACK

In a Flight Reservation system, a Fax-order defect was fixed and Closed in release 1 — then resurfaced in release 2. A Closed defect can be Reopened. Closed is a status, not a promise.

- 1 Five defect families — Functional, Performance, Usability, Compatibility, Security — each discovered by its own kind of testing.
- 2 Classifying by nature routes each bug to the right specialist team — a taxonomy is an assignment mechanism, not paperwork.
- 3 Heartbleed: one missing bounds check, undetected 2+ years, ~17% of secure servers exposed — the cost of untested critical code.
- 4 The defect life cycle: 8 main-path states (New → Closed), the Reopened loop, and 4 side exits (Duplicate, Rejected, Deferred, Not a Bug).
- 5 Before any code is written, three questions filter the queue: valid? in scope? already raised?